

UNIVERSITÉ DE TOULOUSE
Master BI/BEE – Mention Bioinformatique et Génomique
Environnementale

Analyse et Manipulation de la Gene Ontology

*Modélisation de graphe
de *Rattus norvegicus**

Auteur :
Florent LE QUELLEC

Cadre du projet :
UE Algorithmique des
Graphes

Année universitaire 2024–2025



Licence **Creative Commons BY–NC 4.0**

Table des matières

1	Contexte	1
2	Analyse	1
3	Conception	1
3.1	Représentation des données	1
3.2	Fonctionnalités développées	2
3.3	Module spécifique Gene Ontology	3
3.4	Algorithmes	3
3.4.1	Algorithme : GOTerms	3
3.4.2	Algorithme : GeneProducts	4
3.4.3	Algorithme : Max Depth (Optimisé)	5
4	Résultats	7
5	Discussion	7
6	Bilan et Perspectives	7
7	Section spéciale : Méthodologie de travail	8
A	Annexes : Codes Sources	10
A.1	Librairie gm.py (GraphMaster)	10
A.2	Module geneontology.py	25

1 Contexte

Ce projet s'inscrit dans le cadre de l'Unité d'Enseignement *Graphes*, dont l'objet est de fournir des méthodes d'analyse pour des volumes de données conséquents. L'objectif principal est le développement de bibliothèques Python permettant la modélisation et l'exploration de la *Gene Ontology* (GO). Ce projet bioinformatique vise à standardiser et structurer la description des gènes et des produits géniques à travers les différents organismes.

Pour cette étude, nous avons sélectionné *Rattus norvegicus*, communément appelé rat brun. Organisme modèle incontournable dans la communauté scientifique, il est, à l'instar de la souris, prédominant dans les expérimentations *in vivo*, notamment en pharmacologie et en recherche biomédicale. En conséquence, son génome est extrêmement bien documenté ; il fut d'ailleurs l'un des premiers mammifères à être entièrement séquencé en 2004.

2 Analyse

Notre analyse repose sur l'exploitation conjointe de deux sources de données :

- Le fichier **OBO** : il contient les informations structurelles du graphe, à savoir les termes *Gene Ontology* et leurs relations (arcs).
- Le fichier **GOA** : il contient les annotations spécifiques à *Rattus norvegicus*, liant les produits de gènes aux termes du fichier OBO.

Il nous faut donc d'une part pouvoir analyser et modéliser des graphes de manière générique, et d'autre part disposer d'outils pour traiter et parser les fichiers spécifiques à la GO. Pour répondre à ces besoins, nous allons développer deux modules distincts : la bibliothèque *gm.py* (GraphMaster) pour la gestion mathématique des graphes, et le module *geneontology.py* pour l'exploitation des données biologiques.

3 Conception

3.1 Représentation des données

Au vu de la nature hiérarchique et interconnectée de nos données, le choix le plus pertinent est une représentation sous forme de **graphe orienté**. Dans ce modèle :

- Les **nœuds** représentent soit des termes de l'ontologie (*GOTerm*), soit des produits géniques (*GeneProduct*).

- Les **arcs** représentent les relations sémantiques (orientées de l'enfant vers le parent pour *is_a* et *part_of*) ou les annotations (du gène vers le terme).

Pour mettre en œuvre cette architecture, nous avons conçu deux librairies Python distinctes : la première dédiée à la manipulation générique de graphes, et la seconde spécialisée dans le traitement des données de la *Gene Ontology* (GO).

3.2 Fonctionnalités développées

L'ensemble du projet a été réalisé avec Python 3 et les librairies standards telles que `re`, `sys` et `os`. Cependant, deux librairies tierces sont nécessaires : `pandas` et `polars`. Elles sont utilisées au sein de la fonction `read_delim` pour optimiser les performances de lecture.

Dans notre première librairie, nommée *gm.py* (GraphMaster), nous avons implémenté une classe *graph* qui encapsule les informations suivantes :

- Le dictionnaire des nœuds et leurs attributs.
- Le dictionnaire des arcs (liste d'adjacence) et leurs poids éventuels.
- Le caractère dirigé (orienté) ou non du graphe.

Les méthodes suivantes ont été implémentées pour manipuler cette structure :

- *add_node* / *add_edge* : Permettent l'ajout dynamique de nœuds et d'arcs.
- *node_exists* / *edge_exists* : Vérifient la présence d'un élément dans le graphe.
- *nodes* : Renvoie la liste triée de tous les nœuds.
- *nb_nodes* / *nb_edges* : Renvoient respectivement le nombre total de sommets et d'arêtes.
- *neighbors* : Renvoie la liste des voisins (successeurs) d'un nœud donné.
- *predecessors* : Renvoie la liste des prédécesseurs d'un nœud (utile pour remonter les relations parents-enfants).
- *read_delim* : Fonction utilitaire pour charger un graphe depuis un fichier texte délimité (CSV/TXT).
- *BFS* (Breadth-First Search) : Parcours en largeur, permettant notamment de calculer les plus courts chemins depuis un nœud source.
- *DFS* (Depth-First Search) : Parcours en profondeur, utilisé pour l'exploration exhaustive des branches.
- *connected_components* : Identifie les sous-graphes disjoints (pour les graphes non orientés).
- *edges_filter* : Crée un sous-graphe en filtrant les arcs selon un critère de poids.
- *clustering_coefficient* : Calcule le coefficient de clustering (transitivité locale) d'un nœud.

- *is_acyclic* : Vérifie si le graphe est un DAG (Directed Acyclic Graph).
- *topological_sort* : Propose un ordre linéaire des nœuds respectant la direction des arcs.

3.3 Module spécifique Gene Ontology

70 Notre seconde librairie, *geneontology.py*, utilise les fonctions de *gm.py* pour manipuler les données biologiques. Elle implémente les fonctions métiers suivantes :

- *load_OBO* : Parse le fichier de structure de l'ontologie (.obo). Cette fonction gère les références anticipées (*forward references*).
- *load_GOA* : Charge le fichier d'annotations (.goa) et lie les produits géniques aux termes GO correspondants.
- *GOTerms* : Récupère les termes GO associés à un produit génique donné (avec récursion optionnelle).
- *GeneProducts* : Récupère les produits géniques associés à un terme GO (avec récursion optionnelle).
- 80 — *max_depth* : Calcule la profondeur maximale des trois sous-ontologies (BP, MF, CC) en utilisant une optimisation par index inversé.

3.4 Algorithmes

3.4.1 Algorithme : GOTerms

Cet algorithme décrit la remontée vers les ancêtres (parents) pour trouver tous les termes associés à un gène.

Algorithme 1 : Récupération des termes GO d'un produit génique

Entrées Graphe $G = (V, E)$, Identifiant Gène g , Booléen *recursif*

Sortie : Ensemble des termes GO associés T

```
1  $T \leftarrow \text{VoisinsSortants}(G, g);$ 
2 if recursif est Vrai then
3    $File \leftarrow \text{Liste}(T);$ 
4    $Visites \leftarrow T;$ 
5   while  $File$  n'est pas vide do
6      $u \leftarrow \text{Defiler}(File);$ 
7      $Parents \leftarrow \text{VoisinsSortants}(G, u);$            // Relation is_a
8     foreach  $p \in Parents$  do
9       if  $p \notin Visites$  then
10        Ajouter  $p$  à  $T$ ;
11        Ajouter  $p$  à  $Visites$ ;
12        Enfiler  $p$  dans  $File$ ;
13 return  $T$ ;
```

3.4.2 Algorithme : GeneProducts

Ici, on décrit la descente vers les termes plus spécifiques (descendants) pour agglomérer tous les gènes concernés.

Algorithme 2 : Récupération des gènes associés à un terme GO

Entrées Graphe $G = (V, E)$, Identifiant Terme t_{racine} , Booléen *recursif*

Sortie : Ensemble des gènes P

```
1 TermesCibles  $\leftarrow \{t_{racine}\};$ 
  /* Étape 1 : Expansion des termes (Descente vers les
    enfants) */
2 if recursif est Vrai then
3   File  $\leftarrow \{t_{racine}\};$ 
4   Visites  $\leftarrow \{t_{racine}\};$ 
5   while File n'est pas vide do
6      $u \leftarrow \text{Depiler}(\text{File});$ 
7     Enfants  $\leftarrow \text{Predecesseurs}(G, u);$  // Sens inverse des arcs
8     foreach  $v \in \text{Enfants}$  do
9       if  $\text{Type}(v) == \text{"GOTerm"}$  et  $v \notin \text{Visites}$  then
10        Ajouter  $v$  à TermesCibles;
11        Ajouter  $v$  à Visites;
12        Empiler  $v$  dans File;
  /* Étape 2 : Collecte des gènes annotés */
13  $P \leftarrow \emptyset;$ 
14 foreach  $t \in \text{TermesCibles}$  do
15   Sources  $\leftarrow \text{Predecesseurs}(G, t);$ 
16   foreach  $s \in \text{Sources}$  do
17     if  $\text{Type}(s) == \text{"GeneProduct"}$  then
18       Ajouter  $s$  à  $P$ ;
19 return  $P$ ;
```

3.4.3 Algorithme : Max Depth (Optimisé)

Cet algorithme détaille la phase de pré-calcul (index inversé) et la récursion avec mémoïsation.

Algorithme 3 : Calcul de la Profondeur Maximale (Optimisé)

Entrées Graphe $G = (V, E)$, Liste des Racines $R = \{BP, MF, CC\}$

Sortie : Dictionnaire des profondeurs D

```
/* 1. Pré-calcul de l'index inversé (Optimisation
   complexité) */
```

1 *IndexEnfants* $\leftarrow$ DictionnaireVide();2 **foreach** $Arc(u, v) \in E$ **do**

```
3 | Ajouter  $u$  à  $IndexEnfants[v]$ ; //  $u$  est enfant de  $v$ 
```

```
4 Memo ← DictionnaireVide();
```

```
/* 2. Définition de la fonction récursive interne */
```

5 Fonction Hauteur (u) :

6 | **if** $u \in Memo$ **then**

```

7      |      | return Memo[u];

```

8	$Candidats \leftarrow IndexEnfants[u];$
---	---

9	$EnfantsValides \leftarrow \emptyset;$
---	--

```

10  foreach  $c \in Candidates$  do

```

```

11      if  $Type(c) \neq "GeneProduct"$  then

```

12		Ajouter c à $EnfantsValides$;
----	--	----------------------------------

13	if <i>EnfantsValides</i> <i>est vide</i> then
----	---

14		$Memo[u] \leftarrow 0;$
-----------	--	-------------------------

15	return 0;
----	------------------

16	$MaxH \leftarrow 0;$
----	----------------------

17	foreach $enfant \in EnfantsValides$ do
----	--

18	$h \leftarrow \text{Hauteur}(\text{enfant}) ;$
----	--

19	if $h > MaxH$ then
----	----------------------------------

20			$MaxH \leftarrow h;$
----	--	--	----------------------

21	$Memo[u] \leftarrow 1 + MaxH;$
----	--------------------------------

```

22   return  $1 + MaxH$ ;

```

```
/* 3. Exécution principale */
```

```
23 D ← DictionnaireVide();
```

24 **foreach** $racine \in R$ **do**

25 | Vider *Memo*;

26 $D[racine] \leftarrow \text{Hauteur}(racine) ;$

```

27 return  $D$ ;

```


4 Résultats

Les résultats obtenus avec les données de *Rattus norvegicus* sont les suivants :

- **43 328** termes GO chargés.
- **76 581** relations (*is_a*, *part_of*).
- **47 072** gènes (*GeneProducts*) ajoutés au graphe.

100 L'analyse topologique révèle les profondeurs maximales suivantes :

- Biological Process : 18
- Molecular Function : 12
- Cellular Component : 14

Ces résultats confirment la richesse de l'annotation du génome du rat brun, caractérisée par un nombre important de gènes et une grande profondeur dans l'arbre ontologique.

5 Discussion

En analysant les résultats, on remarque un ratio important entre le nombre de gènes et les termes GO. Cela implique une forte redondance : un même terme annote sou-
110 vent de très nombreux gènes, formant ainsi des centres génétiques. Cette densité peut s'expliquer par une surreprésentation de certains gènes dans l'annotation.

En particulier, l'analyse des profondeurs montre que le domaine des *Biological Processes* (BP) est le plus profond. Compte tenu de l'utilisation du rat (*Rattus norvegicus*) comme organisme modèle en médecine, on peut supposer un biais d'étude vers les mécanismes pathologiques (cancer, maladies humaines). À l'inverse, la *Molecular Function*, moins sujette à ces contextes physiopathologiques complexes, présente logiquement la profondeur la plus faible des trois domaines.

6 Bilan et Perspectives

120 Les besoins initiaux du projet ont été satisfaits. Les modules développés permettent de charger efficacement les données de la *Gene Ontology*, d'obtenir les statistiques fondamentales (nombre de termes GO, produits géniques chargés) et d'effectuer des calculs topologiques complexes comme la profondeur des sous-ontologies.

Le choix de Python, langage de haut niveau, a facilité l'implémentation des structures de données. Si cette accessibilité a permis un développement rapide, certains algorithmes se sont révélés coûteux en temps de calcul sur des volumes de données importants, nécessitant une optimisation (indexation).

130 Les bibliothèques produites restent perfectibles, notamment concernant la visualisation graphique des réseaux qui n'a pas été abordée ici. Néanmoins, ce travail constitue une base fonctionnelle solide pour mener des études d'annotation fonctionnelle, de phylogénie, d'analyse médicale ou encore de génomique comparative.

7 Section spéciale : Méthodologie de travail

Dans le cadre de ce projet, j'ai travaillé en autonomie pour l'analyse, la conception et l'implémentation des modules. J'ai également consulté les supports de cours ainsi que la documentation officielle de la *Gene Ontology* pour comprendre la structure des fichiers OBO.

Conformément aux consignes, j'ai eu recours à l'intelligence artificielle générative (Gemini) comme assistant de développement. Ce recours a été transparent et s'est limité à trois aspects spécifiques :

- 140 — **Débogage syntaxique** : Aide à la correction d'erreurs Python et à la gestion des exceptions lors du parsing des fichiers textes.
- **Optimisation algorithmique** : C'est sur ce point que l'aide a été la plus cruciale. Lors du développement de la fonction `max_depth`, la première version de l'algorithme demandait trop de ressources (temps de calcul excessif lié à une complexité quadratique). Gemini m'a aidé et a proposé une solution basée sur un pré-calcul des relations (index inversé).
- **Rédaction technique et orthographique** : Assistance pour la formalisation des algorithmes en pseudo-code (utilisés dans ce rapport), la vérification de la clarté des docstrings et la correction du rapport.

150 J'ai configuré l'assistant avec une consigne imposant des contraintes pédagogiques strictes :

- **Interdiction de fournir du code source** : L'IA avait pour consigne explicite de « *ne jamais fournir de code directement* » mais exclusivement du pseudocode ou des descriptions algorithmiques, m'obligeant à traduire moi-même la logique en Python.

- **Rôle d'expert-conseil** : L'assistant a été paramétré pour agir en tant qu'« *expert en bioinformatique spécialisé dans la théorie des graphes* », avec pour but de guider la conception sans donner la solution finale.
- **Ancrage dans le cours** : Les explications devaient faire le lien avec les concepts théoriques du cours et les structures de données du TP (listes d'adjacence, dictionnaires), assurant une certaine cohérence avec l'enseignement.

160

A Annexes : Codes Sources

A.1 Librairie gm.py (GraphMaster)

```
1  #!/usr/bin/env python
2  # -*- coding: utf-8 -*-
3  """
4  Graph Manipulation Library (gm.py)
5  =====
6  Auteurs : Florent LE QUELLEC
7
8  Cette bibliothèque fournit un ensemble de fonctions pour manipuler des
9  ↪ graphes
10 représentés sous forme de dictionnaires Python. Compatible avec des
11 ↪ graphes
12 dirigés ou non dirigés, pondérés ou non, et intégrable dans des workflows
13 bioinformatiques ou analytiques légers.
14
15 Dépendances : polars, pandas
16 """
17
18 import polars as pl
19 import pandas as pd
20 from pprint import pprint
21
22 class graph:
23     """
24     Classe représentant un graphe (orienté ou non, pondéré ou non).
25     Structure de données : Liste d'adjacence via des dictionnaires
26     ↪ imbriqués.
27     """
28
29     def __init__(self, directed=True, weighted=False,
30     ↪ weight_attribute=None):
31         """
32         Initialise un graphe vide.
33
34         Parameters
35         -----
36         directed : bool, optional
37             Indique si le graphe est dirigé (True) ou non dirigé (False).
38             ↪ Par défaut True.
39         weighted : bool, optional
40             Indique si le graphe possède des poids d'arêtes. Par défaut
41             ↪ False.
42         weight_attribute : str, optional
```

```

37         Nom de l'attribut de poids si applicable (ex: 'weight',
38         ↪ 'distance').
39     """
40     self.nodes = {} # Stocke les nœuds et leurs attributs : {id:
41     ↪ {attr: val}}
42     self.edges = {} # Stocke la structure : {source: {cible: {attr:
43     ↪ val}}}
44     self.directed = directed
45     self.weighted = weighted
46     self.weight_attribute = weight_attribute
47
48 def __str__(self):
49     """Représentation textuelle du graphe."""
50     lines = [
51         f"Graphe {'dirigé' if self.directed else 'non dirigé'}",
52         f"Nombre de nœuds : {len(self.nodes)}",
53         f"Nombre d'arêtes : {self.nb_edges()}",
54         "---- Aperçu des arêtes ----"
55     ]
56     # On limite l'affichage pour éviter de saturer la console sur les
57     ↪ gros graphes
58     count = 0
59     for u, targets in self.edges.items():
60         for v, attrs in targets.items():
61             if count < 10:
62                 lines.append(f" {u} -> {v} {attrs}")
63                 count += 1
64             if count >= 10:
65                 lines.append(f" ... (+ {count - 10} autres arêtes)")
66     return "\n".join(lines)
67
68 def node_exists(self, n):
69     """
70     Vérifie si un nœud existe dans le graphe.
71
72     Parameters
73     -----
74     n : str or int
75         Identifiant du nœud à tester.
76
77     Returns
78     -----
79     bool
80         True si le nœud existe, False sinon.
81     """
82     return n in self.nodes
83
84 def add_node(self, node_id, attributes=None):

```

```

81         """
82         Ajoute un nœud au graphe s'il n'existe pas déjà.
83
84         Parameters
85         -----
86         node_id : str or int
87             Identifiant du nœud à ajouter.
88         attributes : dict, optional
89             Dictionnaire d'attributs associés au nœud.
90
91         Returns
92         -----
93         dict
94             Dictionnaire des attributs du nœud ajouté (ou existant).
95         """
96         if node_id not in self.nodes:
97             self.nodes[node_id] = attributes or {}
98             self.edges[node_id] = {} # initialise le dictionnaire des
99                                     ↪ voisins
100         return self.nodes[node_id]
101
102 def edge_exists(self, n1, n2):
103     """
104     Vérifie si une arête entre deux nœuds existe.
105
106     Parameters
107     -----
108     n1 : str or int
109         Nœud source.
110     n2 : str or int
111         Nœud cible.
112
113     Returns
114     -----
115     bool
116         True si l'arête (n1, n2) existe, False sinon.
117     """
118     return n1 in self.edges and n2 in self.edges[n1]
119
120 def add_edge(self, node_id1, node_id2, attributes=None):
121     """
122     Ajoute une arête entre deux nœuds. Crée les nœuds automatiquement
123     ↪ s'ils manquent.
124
125     Si le graphe est non dirigé, l'arête est ajoutée dans les deux
126     ↪ sens
127     (node_id1 -> node_id2 et node_id2 -> node_id1) partageant les
128     ↪ mêmes attributs.

```

```

125
126     Parameters
127     -----
128     node_id1 : str or int
129         Identifiant du nœud source.
130     node_id2 : str or int
131         Identifiant du nœud cible.
132     attributes : dict, optional
133         Dictionnaire d'attributs de l'arête.
134
135     Returns
136     -----
137     dict
138         Dictionnaire des attributs de l'arête ajoutée.
139     """
140     self.add_node(node_id1)
141     self.add_node(node_id2)
142
143     # On évite d'écraser une arête existante sauf si nécessaire
144     if not self.edge_exists(node_id1, node_id2):
145         attr = attributes or {}
146         self.edges[node_id1][node_id2] = attr
147
148         if not self.directed:
149             # Partage de la référence du dictionnaire d'attributs
150             self.edges[node_id2][node_id1] =
151                 ↪ self.edges[node_id1][node_id2]
152
153     return self.edges[node_id1][node_id2]
154
155 def get_nodes(self):
156     """
157     Renvoie la liste triée des identifiants de nœuds du graphe.
158     (Renommé de 'nodes' pour éviter la confusion avec l'attribut
159     ↪ self.nodes).
160
161     Returns
162     -----
163     list
164         Liste triée des clés des nœuds.
165     """
166     return sorted(self.nodes.keys())
167
168 def nb_nodes(self):
169     """
170     Renvoie le nombre de nœuds du graphe.
171
172     Returns

```

```

171         -----
172         int
173         Nombre de nœuds.
174         """
175         return len(self.nodes)
176
177     def nb_edges(self):
178         """
179         Renvoie le nombre d'arêtes du graphe.
180
181         Pour les graphes non dirigés, divise le total par 2 car chaque
182         ↔ arête
183         est stockée deux fois (u->v et v->u).
184
185         Returns
186         -----
187         int
188         Nombre d'arêtes.
189         """
190         count = sum(len(v) for v in self.edges.values())
191         return count // 2 if not self.directed else count
192
193     def neighbors(self, node_id):
194         """
195         Renvoie la liste des voisins (successeurs) d'un nœud donné.
196
197         Parameters
198         -----
199         node_id : str or int
200             Identifiant du nœud.
201
202         Returns
203         -----
204         list
205             Liste des identifiants de nœuds voisins.
206         """
207         if node_id in self.edges:
208             return list(self.edges[node_id].keys())
209         return []
210
211     def predecessors(self, node_id):
212         """
213         Renvoie la liste des nœuds qui pointent vers node_id.
214         (Parents dans un arbre, ou prédécesseurs dans un graphe orienté).
215
216         Parameters
217         -----
218         node_id : str or int

```



```

218         Identifiant du nœud cible.
219
220     Returns
221     -----
222     list
223         Liste des prédécesseurs.
224     """
225     # Si le graphe n'est pas orienté, voisins = prédécesseurs
226     if not self.directed:
227         return self.neighbors(node_id)
228
229     preds = []
230     # On parcourt tous les nœuds sources possibles
231     for u in self.edges:
232         if node_id in self.edges[u]:
233             preds.append(u)
234     return preds
235
236     def edges_tuples(self):
237         """
238         Renvoie la liste de toutes les arêtes sous forme de tuples
239         ↪ (source, cible).
240
241         Returns
242         -----
243         list of tuple
244             Liste de tuples (u, v).
245         """
246         return [(u, v) for u in self.nodes for v in self.neighbors(u)]
247
248     @classmethod
249     def read_delim(cls, filename, column_separator='\t', directed=True,
250     ↪ weighted=False, weight_attribute=None):
251         """
252         Lit un fichier délimité (ex: TSV, CSV) et construit un objet graph.
253
254         Les deux premières colonnes représentent les nœuds connectés
255         ↪ (Source, Target),
256         les suivantes contiennent les attributs d'arêtes.
257
258         Parameters
259         -----
260         filename : str
261             Chemin vers le fichier à lire.
262         column_separator : str, optional
263             Caractère de séparation (par défaut : tabulation).
264         directed : bool, optional
265             Indique si le graphe doit être dirigé.

```

```

263     weighted : bool, optional
264         Indique si le graphe doit être pondéré.
265     weight_attribute : str, optional
266         Nom de l'attribut de poids.
267
268     Returns
269     -----
270     graph
271         Instance de la classe graph remplie.
272     """
273     # Lecture optimisée avec Polars
274     try:
275         df = pl.read_csv(
276             filename,
277             separator=column_separator,
278             has_header=True,
279             infer_schema_length=1000,
280             quote_char=None,
281             truncate_ragged_lines=True
282         )
283     except Exception as e:
284         print(f"Erreur lors de la lecture du fichier : {e}")
285         return cls()
286
287     cols = df.columns
288     if len(cols) < 2:
289         raise ValueError("Le fichier doit contenir au moins deux
290             ↪ colonnes (source, target).")
291
292     src_col, tgt_col = cols[0], cols[1]
293     att_cols = cols[2:]
294
295     # Création de l'instance
296     g = cls(directed=directed, weighted=weighted,
297         ↪ weight_attribute=weight_attribute)
298
299     # Conversion en pandas pour l'itération (souvent plus simple pour
300     ↪ les lignes mixtes)
301     pdf = df.to_pandas()
302
303     for _, row in pdf.iterrows():
304         u = row[src_col]
305         v = row[tgt_col]
306         # Création du dictionnaire d'attributs pour l'arête
307         att = {col: row[col] for col in att_cols}
308         g.add_edge(u, v, att)
309
310     return g

```

```

308
309 def BFS(self, s, cible=None):
310     """
311     Algorithme de parcours en largeur (Breadth-First Search).
312     Calcule les plus courts chemins (en nombre d'arêtes) depuis une
313     ↪ source.
314
315     Parameters
316     -----
317     s : str or int
318     Sommet de départ.
319     cible : str or int, optional
320     Sommet d'arrivée. Si spécifié, l'algorithme s'arrête dès qu'il
321     ↪ est trouvé.
322
323     Returns
324     -----
325     dict
326     Si cible est trouvée : {'Distance': int, 'chemin': list,
327     ↪ 'source': s}
328     Sinon (parcours complet) : {'etats': dict, 'distances': dict,
329     ↪ 'parents': dict, 'source': s}
330     """
331     if s not in self.nodes:
332         print(f"Erreur : Le nœud de départ {s} n'existe pas.")
333         return {}
334
335     # Initialisation
336     etats = {u: 'blanc' for u in self.nodes}
337     distances = {u: float('inf') for u in self.nodes}
338     parents = {u: None for u in self.nodes}
339
340     etats[s] = 'gris' # En cours de visite
341     distances[s] = 0
342     attente = [s] # File (FIFO)
343
344     while attente:
345         u = attente.pop(0)
346
347         if cible and u == cible:
348             break
349
350         for voisin in self.neighbors(u):
351             if etats[voisin] == 'blanc':
352                 etats[voisin] = 'gris'
353                 distances[voisin] = distances[u] + 1
354                 parents[voisin] = u
355                 attente.append(voisin)

```

```

352
353         etats[u] = 'noir' # Visité
354
355     # Reconstruction du chemin si une cible était demandée
356     if cible:
357         if parents[cible] is not None or cible == s:
358             chemin = [cible]
359             curr = cible
360             while curr != s:
361                 curr = parents[curr]
362                 chemin.append(curr)
363             chemin.reverse()
364             return {"Distance": distances[cible], "chemin": chemin,
365                    ↪ "source": s}
366         else:
367             return {"Distance": float('inf'), "chemin": [], "source":
368                    ↪ s, "error": "Cible inatteignable"}
369
370     return {"etats": etats, "distances": distances, "parents":
371            ↪ parents, "source": s}
372
373 def connected_components(self):
374     """
375     Identifie les composantes connexes d'un graphe NON orienté via BFS.
376
377     Returns
378     -----
379     dict
380         Dictionnaire {node_id: component_id}.
381     """
382     if self.directed:
383         print("Attention : connected_components est conçu pour les
384               ↪ graphes non orientés.")
385         # On continue quand même, mais le résultat représente des
386         ↪ composantes faiblement connexes
387         # si on ignore la direction, ou juste l'accessibilité si on la
388         ↪ garde.
389
390     n_CC = 0
391     CC = {u: None for u in self.nodes}
392
393     for u in self.nodes:
394         if CC[u] is None:
395             # On lance un BFS complet depuis ce nœud non visité
396             parcours = self.BFS(u)
397
398             # Tous les nœuds atteints appartiennent à la composante
399             ↪ n_CC

```

```

393         # Le BFS retourne 'distances' pour les nœuds visités
394         visited_nodes = [n for n, d in
395             ↪ parcours['distances'].items() if d != float('inf')]
396
397         for v in visited_nodes:
398             CC[v] = n_CC
399             n_CC += 1
400         return CC
401
402     def sousgraphe_induit(self, nodes_subset):
403         """
404         Crée un sous-graphe induit par une liste de nœuds donnée.
405
406         Parameters
407         -----
408         nodes_subset : list
409             Liste des identifiants de nœuds à conserver.
410
411         Returns
412         -----
413         graph
414             Nouveau graphe contenant uniquement les nœuds spécifiés et les
415             ↪ arêtes entre eux.
416         """
417         sg = graph(
418             directed=self.directed,
419             weighted=self.weighted,
420             weight_attribute=self.weight_attribute
421         )
422
423         # Ajout des nœuds
424         for u in nodes_subset:
425             if u in self.nodes:
426                 sg.add_node(u, self.nodes[u])
427
428         # Ajout des arêtes existantes entre ces nœuds
429         for u in nodes_subset:
430             if u in self.edges:
431                 for v, attrs in self.edges[u].items():
432                     if v in nodes_subset:
433                         sg.add_edge(u, v, attrs)
434
435         return sg
436
437     def edges_filter(self, attribut, seuil):
438         """
439         Filtre le graphe pour ne garder que les arêtes respectant un
440         ↪ critère numérique.

```

```

438
439     Parameters
440     -----
441     attribut : str
442         Nom de l'attribut de l'arête à tester (ex: 'weight').
443     seuil : float
444         Valeur minimale pour conserver l'arête.
445
446     Returns
447     -----
448     graph
449         Nouveau graphe filtré.
450     """
451     G_filtre = graph(directed=self.directed, weighted=self.weighted)
452
453     # On copie tous les nœuds
454     for node_id, attrs in self.nodes.items():
455         G_filtre.add_node(node_id, attrs)
456
457     # On ne copie que les arêtes valides
458     for u in self.edges:
459         for v, attrs in self.edges[u].items():
460             if attribut in attrs and attrs[attribut] >= seuil:
461                 G_filtre.add_edge(u, v, attrs)
462     return G_filtre
463
464     def clustering_coefficient(self):
465         """
466         Calcule le coefficient de clustering local pour chaque nœud.
467         C = (nombre de liens entre voisins) / (nombre de liens possibles
468             ↪ entre voisins).
469
470         Returns
471         -----
472         dict
473             {node_id: coefficient (float entre 0 et 1)}
474         """
475         coeffs = {}
476         for node_id in self.nodes:
477             voisins = self.neighbors(node_id)
478             k = len(voisins)
479
480             # Si moins de 2 voisins, pas de connexions possibles entre eux
481             ↪ -> coeff = 0
482             if k < 2:
483                 coeffs[node_id] = 0.0
484                 continue

```

```

484         liens_reels = 0
485         # On regarde toutes les paires de voisins uniques
486         for v1 in voisins:
487             for v2 in voisins:
488                 if v1 != v2:
489                     if self.edge_exists(v1, v2):
490                         liens_reels += 1
491
492         # Nombre max de liens possibles entre k voisins
493         # k * (k-1) pour dirigé, k * (k-1) / 2 pour non dirigé.
494         # Ici l'algo compte v1->v2 et v2->v1 séparément dans la
495             ↪ boucle,
496         # donc on divise par k*(k-1) dans tous les cas pour
497             ↪ normaliser.
498         coeffs[node_id] = liens_reels / (k * (k - 1))
499
500     return coeffs
501
502 def DFS(self):
503     """
504     Algorithme de parcours en profondeur (Depth-First Search).
505     Utilisé pour la classification des arêtes, la détection de cycles
506         ↪ et le tri topologique.
507
508     Returns
509     -----
510     dict
511         Structure contenant l'état du parcours, les temps de
512             ↪ découverte/fin,
513         la classification des arêtes et l'ordre de fin.
514     """
515     a = {
516         'etat': {},
517         'parents': {},
518         'decouvert': {}, # Timestamp de début
519         'fin': {}, # Timestamp de fin
520         'classification': {}, # Types d'arêtes
521         'ordre_fin': [], # Liste pour tri topologique
522         'temps': 0
523     }
524
525     # Initialisation
526     for u in self.nodes:
527         a['etat'][u] = 'inexplored'
528         a['parents'][u] = None
529
530     # Boucle principale pour traiter les composantes disjointes
531     for u in self.nodes:

```

```

528         if a['etat'][u] == 'inexploire':
529             self._DFSvisite(u, a)
530         return a
531
532     def _DFSvisite(self, u, a):
533         """Fonction interne récursive pour le DFS."""
534         a['etat'][u] = 'decouvert'
535         a['temps'] += 1
536         a['decouvert'][u] = a['temps']
537
538         for v in self.neighbors(u):
539             if a['etat'][v] == 'inexploire':
540                 a['parents'][v] = u
541                 a['classification'][(u, v)] = 'branche'
542                 self._DFSvisite(v, a)
543             elif a['etat'][v] == 'decouvert':
544                 # Arête vers un ancêtre non terminé -> Cycle détecté
545                 a['classification'][(u, v)] = 'retour'
546             elif a['decouvert'][u] < a['decouvert'][v]:
547                 a['classification'][(u, v)] = 'arete avant'
548             else:
549                 a['classification'][(u, v)] = 'transversale' # Cross edge
550
551         a['etat'][u] = 'traite'
552         a['temps'] += 1
553         a['fin'][u] = a['temps']
554
555         # Ajout à la liste pour le tri topologique (post-ordre)
556         a['ordre_fin'].append(u)
557         return a
558
559     def is_acyclic(self):
560         """
561         Vérifie si le graphe est acyclique (DAG).
562
563         Returns
564         -----
565         bool
566         True si acyclique, False si contient un cycle.
567         """
568         # Un cycle existe ssi le DFS trouve une arête 'retour'
569         res = self.DFS()
570         for type_arete in res['classification'].values():
571             if type_arete == 'retour':
572                 return False
573         return True
574
575     def topological_sort(self):

```



```

576         """
577         Effectue un tri topologique des nœuds (uniquement pour les DAG).
578
579         Returns
580         -----
581         list or None
582             Liste des nœuds ordonnée linéairement, ou None si cycle
583             ↪ détecté.
584         """
585         if not self.is_acyclic():
586             print("Erreur : Le graphe contient un cycle, tri topologique
587                   ↪ impossible.")
588             return None
589
590         res = self.DFS()
591         # L'ordre topologique est l'inverse de l'ordre de fin de traitement
592         return res['ordre_fin'][::-1]
593
594 ##### Tests unitaires basiques #####
595 if __name__ == "__main__":
596     print("# Graph lib tests")
597
598     print("\n## 1. Création et affichage")
599     g = graph(directed=True)
600     g.add_node('A')
601     g.add_node('B')
602     g.add_edge('A', 'B', {'weight': 5})
603     print(g)
604
605     print("\n## 2. Graphe non dirigé")
606     g2 = graph(directed=False)
607     g2.add_edge('A', 'B')
608     g2.add_edge('A', 'C')
609     g2.add_edge('B', 'D')
610     g2.add_edge('C', 'E')
611     # Création d'un cycle pour tester is_acyclic plus tard (A-B-D-A
612     # ↪ impossible direct mais A-B-A oui en non dirigé)
613     # Pour le test clustering
614     g2.add_edge('B', 'C')
615
616     print(g2)
617     print(f"Nombre de noeuds: {g2.nb_nodes()}")
618     print(f"Nombre d'arêtes: {g2.nb_edges()}")
619
620     print("\n## 3. BFS (Plus court chemin)")
621     # Chemin A -> E
622     res_bfs = g2.BFS('A', 'E')

```

```

621 print("Chemin A -> E :", res_bfs.get('chemin'))
622 print("Distance :", res_bfs.get('Distance'))
623
624 print("\n## 4. Clustering Coefficient")
625 cc = g2.clustering_coefficient()
626 print("Coefficients de clustering :", cc)
627
628 print("\n## 5. DFS et Tri Topologique (Sur un DAG)")
629 dag = graph(directed=True)
630 dag.add_edge('Chemise', 'Ceinture')
631 dag.add_edge('Chemise', 'Cravate')
632 dag.add_edge('Cravate', 'Veste')
633 dag.add_edge('Ceinture', 'Veste')
634 dag.add_edge('Pantalon', 'Ceinture')
635 dag.add_edge('Pantalon', 'Chaussures')
636 dag.add_edge('Chaussettes', 'Chaussures')
637
638 print("Est acyclique ?", dag.is_acyclic())
639 print("Tri topologique (Ordre d'habillage) :", dag.topological_sort())
640
641 print("\n## 6. Détection de cycle")
642 cycle_g = graph(directed=True)
643 cycle_g.add_edge('A', 'B')
644 cycle_g.add_edge('B', 'C')
645 cycle_g.add_edge('C', 'A') # Cycle
646 print("Graphe cyclique est acyclique ?", cycle_g.is_acyclic())

```

A.2 Module geneontology.py

```
1  #!/usr/bin/env python
2  # -*- coding: utf-8 -*-
3  """
4  Gene Ontology Analysis Module (geneontology.py)
5  =====
6  Auteurs : Florent LE QUELLEC
7
8  Ce module permet de charger et de manipuler les données de la Gene
9  ↪ Ontology (GO).
10 Il construit un graphe orienté acyclique (DAG) à partir des fichiers de
11 ↪ définition
12 (.obo) et y associe les annotations géniques (.goa/.gaf).
13
14 Fonctionnalités principales :
15 - Parsing robuste des fichiers OBO (gestion des forward references).
16 - Parsing des fichiers d'annotations (GOA).
17 - Exploration de la hiérarchie (Ancêtres / Descendants).
18 - Calcul optimisé de la profondeur des ontologies.
19
20 Dépendances :
21 - gm (Module de gestion de graphe local)
22 - re (Expressions régulières)
23 """
24
25 import re
26 import sys
27 import os
28 import gm
29
30 # --- Visualisation conceptuelle ---
31 # Le graphe GO est un DAG (Directed Acyclic Graph).
32 # Les arcs 'is_a' vont de l'Enfant vers le Parent (Spécifique ->
33 ↪ Générique).
34 #
35
36 def load_OBO(filename='go-basic.obo') :
37     """
38     Parse un fichier OBO et construit un graphe de termes GO.
39
40     Cette fonction gère les références anticipées ("forward references") :
41     si un terme enfant fait référence à un parent qui n'a pas encore été
42     lu dans le fichier, le parent est créé à la volée (noeud squelette)
43     pour garantir la connectivité du graphe.
44
45     Parameters
```

```

43  -----
44  filename : str, optional
45      Chemin vers le fichier .obo. Par défaut 'go-basic.obo'.
46
47  Returns
48  -----
49  gm.graph
50      Le graphe orienté contenant les termes GO.
51      Structure des arêtes : Enfant -> Parent (relations 'is_a',
52      ↪ 'part_of').
53      """
54  go_graph = gm.graph(directed=True, weighted=False)
55  go_graph.alt_id = {} # Stockage des IDs alternatifs pour redirection
56
57  def parseTerm(lines):
58      """Fonction interne pour parser un bloc [Term] ligne par ligne."""
59      go_id = None
60      is_obsolete = False
61
62      # 1. Extraction ID et Obsolescence
63      for line in lines:
64          if line.startswith('id:'):
65              # Exemple : "id: GO:0000001" -> on garde "GO:0000001"
66              go_id = line.split()[1]
67          elif line.startswith('is_obsolete: true'):
68              is_obsolete = True
69
70      # On ignore les termes sans ID ou obsolètes
71      if not go_id or is_obsolete:
72          return
73
74      # 2. Création ou Mise à jour du nœud
75      # Note : Le nœud peut déjà exister (créé comme parent par un
76      # ↪ enfant lu avant)
77      if go_id not in go_graph.nodes:
78          go_graph.add_node(go_id, {'type': 'GOTerm'})
79      else:
80          # Si le nœud existait déjà, on confirme son type
81          go_graph.nodes[go_id]['type'] = 'GOTerm'
82
83      go_attr = go_graph.nodes[go_id]
84
85      # 3. Parsing des attributs et relations
86      for line in lines:
87          # Nettoyage des commentaires inline (ex: " ... ! description")
88          if '!' in line:
89              line = line.split('!')[0].strip()

```

```

89         if line.startswith('name:'):
90             go_attr['name'] = line.replace('name:', '').strip()
91         elif line.startswith('namespace:'):
92             go_attr['namespace'] = line.replace('namespace:',
93             ↪ '').strip()
94         elif line.startswith('def:'):
95             # Nettoyage des guillemets autour de la définition
96             go_attr['def'] = line.replace('def:',
97             ↪ '').strip().strip('"')
98         elif line.startswith('alt_id:'):
99             parts = line.split()
100             if len(parts) > 1:
101                 go_graph.alt_id[parts[1]] = go_id
102
103     # --- GESTION DES RELATIONS ---
104     parent_id = None
105     relation = None
106
107     if line.startswith('is_a:'):
108         parts = line.split()
109         if len(parts) >= 2:
110             parent_id = parts[1]
111             relation = 'is_a'
112
113     elif line.startswith('relationship: part_of'):
114         parts = line.split()
115         # On cherche l'élément qui ressemble à un ID GO
116         for p in parts:
117             if p.startswith('GO:'):
118                 parent_id = p
119                 relation = 'part_of'
120                 break
121
122     # Ajout de l'arête avec sécurité "Forward Reference"
123     if parent_id:
124         # Si le parent n'existe pas encore, on le crée (squelette)
125         if parent_id not in go_graph.nodes:
126             go_graph.add_node(parent_id, {'type': 'GOTerm'})
127
128         # Ajout de l'arête : Terme Courant (Enfant) -> Parent
129         go_graph.add_edge(go_id, parent_id, {'relationship':
130         ↪ relation})
131
132     # Lecture principale du fichier
133     try:
134         with open(filename, 'r', encoding='utf-8') as f:
135             buff = []
136             in_term = False

```

```

134         for line in f:
135             line = line.strip()
136             if not line: continue
137
138             if line == '[Term]':
139                 # Si on a fini de lire un terme précédent, on le parse
140                 if buff: parseTerm(buff)
141                 buff = []
142                 in_term = True
143             elif line == '[Typeddef]':
144                 # On arrête le parsing si on tombe sur les définitions
145                 ↪ de types
146                 if buff: parseTerm(buff)
147                 break
148             elif in_term:
149                 buff.append(line)
150             # Ne pas oublier le dernier buffer à la fin du fichier
151             if buff and in_term: parseTerm(buff)
152
153     except FileNotFoundError:
154         print(f"Erreur critique: Le fichier {filename} est introuvable.")
155         return gm.graph()
156
157     return go_graph
158
159 def load_GOA(go, filename, warnings=True):
160     """
161     Parse un fichier GOA (Gene Ontology Annotation) et ajoute les produits
162     ↪ géniques au graphe.
163
164     Le format attendu est le format GAF (Gene Association File).
165     Les arcs créés vont du Gène vers le Terme GO.
166
167     Parameters
168     -----
169     go : gm.graph
170         Le graphe GO chargé précédemment (objet mutable).
171     filename : str
172         Chemin vers le fichier .gaf/.goa.
173     warnings : bool, optional
174         Afficher les avertissements si un terme annoté n'existe pas dans
175         ↪ l'OBO.
176
177     Returns
178     -----
179     None
180         Le graphe est modifié en place.

```

```

179 """
180 # Optimisation : Cache local pour éviter les lookups répétés sur
181 ↪ self.nodes
182 nodes = go.nodes
183 alt_ids = go.alt_id
184
185
186 try:
187     with open(filename, 'r', encoding='utf-8') as f:
188         for line in f:
189             if line.startswith('!'): continue # Ignore les
190             ↪ commentaires
191
192             cols = line.rstrip().split('\t')
193             # Sécurité format GAF (min 15 colonnes, ici on vérifie 11
194             ↪ pour le nécessaire)
195             if len(cols) < 11: continue
196
197             gp_id = cols[1] # DB_Object_ID (Identifiant unique du
198             ↪ gène)
199             gt_id = cols[4] # GO ID (Terme annoté)
200
201             # 1. Résolution d'identifiant alternatif
202             # (Si le fichier d'annotation utilise un vieil ID redirigé
203             ↪ dans l'OBO)
204             if gt_id not in nodes and gt_id in alt_ids:
205                 gt_id = alt_ids[gt_id]
206
207             # 2. Vérification existence du terme
208             if gt_id not in nodes:
209                 if warnings:
210                     # Ce cas arrive si l'OBO est plus vieux que le GOA
211                     ↪ ou incomplet
212                     # print(f" Impossible de rattacher {gp_id} à
213                     ↪ {gt_id} (Terme inconnu)")
214                 pass
215                 continue
216
217             # 3. Création du Produit Génique (s'il n'existe pas
218             ↪ encore)
219             if gp_id not in nodes:
220                 go.add_node(gp_id, {
221                     'id': gp_id,
222                     'type': 'GeneProduct',
223                     'name': cols[2], # Symbol
224                     'desc': cols[9], # Full Name
225                     'aliases': cols[10].split('|')

```

```

219         ))
220
221         # 4. Ajout de l'annotation (Arête GeneProduct -> GOTerm)
222         e_attr = go.add_edge(gp_id, gt_id, {'relationship':
223             ↳ 'annotation'})
224
225         # Ajout du code de preuve (ex: IEA, EXP...) à la liste des
226         ↳ preuves pour cette arête
227         e_attr.setdefault('evidence-codes', []).append(cols[6])
228
229     except FileNotFoundError:
230         print(f"Erreur: Le fichier d'annotation {filename} est
231             ↳ introuvable.")
232
233
234 def GOTerms(go, gp_id, recursive=False):
235     """
236     Retourne les termes GO associés à un produit génique.
237
238     Parameters
239     -----
240     go : gm.graph
241         Le graphe contenant l'ontologie et les annotations.
242     gp_id : str
243         L'identifiant du gène (GeneProduct).
244     recursive : bool, optional
245         Si True, remonte l'arbre pour inclure tous les termes parents
246         ↳ (Ancêtres).
247         Si False, retourne uniquement les annotations directes.
248
249     Returns
250     -----
251     list
252         Liste des identifiants GO trouvés.
253     """
254     if gp_id not in go.nodes:
255         return []
256
257     # Les gènes pointent vers les termes (neighbors directs)
258     termes_initiaux = go.neighbors(gp_id)
259
260     if not recursive:
261         return list(termes_initiaux)
262     else:
263         # Parcours en largeur (BFS) pour remonter vers les parents
264         ↳ (relation is_a)
265         # Note : Dans notre graphe, Edge = Enfant -> Parent, donc
266         ↳ neighbors = Parents.

```



```

261         res = set(termes_initiaux)
262         file_attente = list(termes_initiaux)
263
264         while len(file_attente) != 0:
265             courant = file_attente.pop(0)
266             parents = go.neighbors(courant)
267
268             for parent in parents:
269                 if parent not in res:
270                     res.add(parent)
271                     file_attente.append(parent)
272         return list(res)
273
274
275 def GeneProducts(go, go_id, recursive=False):
276     """
277     Retourne les produits géniques associés à un terme GO.
278
279     Parameters
280     -----
281     go : gm.graph
282         Le graphe.
283     go_id : str
284         L'identifiant du terme GO.
285     recursive : bool, optional
286         Si True, descend dans l'arbre pour inclure les gènes annotés
287         sur des termes enfants (Descendants / Plus spécifiques).
288
289     Returns
290     -----
291     list
292         Liste des identifiants de gènes trouvés.
293     """
294     if go_id not in go.nodes:
295         return []
296
297     termes_cible = {go_id}
298
299     # 1. Si récursif : On descend dans l'arbre pour trouver tous les
300     ↪ termes spécifiques
301     if recursive:
302         file = [go_id]
303         ensemble_visites = {go_id}
304         while len(file) != 0:
305             term = file.pop()
306             # On cherche qui pointe vers nous (Predecessors = Enfants)
307             # Rappel : Edge = Enfant -> Parent
308             entrants = go.predecessors(term)

```

```

308
309         for entrant in entrants:
310             # On ne descend que dans les termes GO (on ne saute pas
311             # ↪ directement aux gènes)
312             # Utilisation de .get() pour sécurité (au cas où un nœud
313             # ↪ n'a pas de type)
314             if go.nodes[entrant].get('type') == 'GOTerm' and entrant
315             # ↪ not in ensemble_visites:
316                 termes_cible.add(entrant)
317                 ensemble_visites.add(entrant)
318                 file.append(entrant)
319
320     # 2. Pour tous les termes identifiés (racine + descendants), on
321     # ↪ récupère les gènes
322     # Les gènes sont des prédécesseurs des termes (Gene -> Terme)
323     genes_trouves = set()
324     for term in termes_cible:
325         sources = go.predecessors(term)
326         for source in sources:
327             if go.nodes[source].get('type') == 'GeneProduct':
328                 genes_trouves.add(source)
329
330     return list(genes_trouves)
331
332
333 def max_depth(go):
334     """
335     Calcule la profondeur maximale des 3 sous-ontologies (BP, MF, CC).
336
337     L'algorithme calcule la "Hauteur" de la racine, définie comme la
338     ↪ longueur
339     du chemin le plus long de la racine vers une feuille (terme le plus
340     ↪ spécifique).
341
342     Optimisation :
343     Utilise un index inversé pré-calculé pour transformer la recherche des
344     ↪ enfants
345     (qui nécessite un scan complet O(N) dans gm.py) en accès direct O(1).
346
347     Parameters
348     -----
349     go : gm.graph
350         Le graphe chargé.
351
352     Returns
353     -----
354     dict
355         {'BP': int, 'MF': int, 'CC': int}

```

```

349     """
350     racines = {
351         'BP': 'GO:0008150', # Biological Process
352         'MF': 'GO:0003674', # Molecular Function
353         'CC': 'GO:0005575' # Cellular Component
354     }
355
356     resultats = {}
357     memo = {}
358
359     # --- OPTIMISATION : Construction de l'index inversé ---
360     # Structure : { Parent : [Liste des Enfants] }
361     # Permet de descendre dans l'arbre (Parent -> Enfant) instantanément.
362     print("    [Optimisation] Construction de l'index inversé...", end='',
363           ↪ flush=True)
364
365     reverse_graph = {}
366     # On parcourt toutes les arêtes. Dans OBO : Source=Enfant,
367     ↪ Cible=Parent.
368     for source_node in go.edges:
369         targets = go.edges[source_node] # targets = parents
370
371         for target_node in targets:
372             if target_node not in reverse_graph:
373                 reverse_graph[target_node] = []
374             reverse_graph[target_node].append(source_node) # target a pour
375             ↪ enfant source
376
377     print(" Fait.")
378     # -----
379
380     def get_height(u):
381         """Fonction récursive avec mémoïsation (Programmation
382         ↪ dynamique)"""
383         if u in memo: return memo[u]
384
385         # Récupération optimisée des enfants
386         enfants = reverse_graph.get(u, [])
387
388         # FILTRAGE : On ne descend que vers d'autres termes GO
389         go_enfants = []
390         for c in enfants:
391             if c in go.nodes:
392                 # On exclut les gènes (qui sont aussi des "enfants" dans
393                 ↪ le sens où ils pointent vers le terme)
394                 # On accepte 'GOTerm' ou None (tolérance) mais pas
395                 ↪ 'GeneProduct'
396                 if go.nodes[c].get('type') != 'GeneProduct':

```

```

391         go_enfants.append(c)
392
393         # Cas de base : Feuille (pas d'enfants termes GO)
394         if not go_enfants:
395             memo[u] = 0
396             return 0
397
398         # Récursion : Hauteur = 1 + max(Hauteur des enfants)
399         try:
400             h = 1 + max(get_height(child) for child in go_enfants)
401         except ValueError:
402             h = 0
403
404         memo[u] = h
405         return h
406
407     # Lancement du calcul pour les 3 racines
408     for nom, root_id in racines.items():
409         if root_id in go.nodes:
410             try:
411                 memo = {} # Reset du cache par sécurité entre les
412                     ↪ ontologies
413                 resultats[nom] = get_height(root_id)
414             except RecursionError:
415                 print(f"Erreur: Profondeur excessive pour {nom} (Cycle
416                     ↪ possible ?)")
417                 resultats[nom] = 0
418         else:
419             resultats[nom] = 0
420
421     return resultats
422
423 ##### Tests unitaires basiques #####
424 if __name__ == "__main__":
425     print("# Gene Ontology module tests")
426
427     # Définition du chemin de test (à adapter selon votre arborescence)
428     # Recherche locale ou dans un sous-dossier 'data'
429     test_paths = [
430         "go-basic.obo",
431         "Python/data/projet/go-basic.obo",
432         "data/go-basic.obo"
433     ]
434
435     filename = None
436     for p in test_paths:
437         if os.path.exists(p):

```

```

437         filename = p
438         break
439
440     if filename:
441         print(f"Chargement de {filename}...")
442         go = load_OBO(filename)
443         print(f"Graph chargé : {len(go.nodes)} noeuds")
444
445         if len(go.nodes) > 0:
446             print("\nCalcul des profondeurs (max_depth)...")
447             depths = max_depth(go)
448             print("Profondeurs :", depths)
449
450             # Vérification de cohérence (CC ne doit pas être 0)
451             if depths.get('CC', 0) == 0:
452                 print("  Avertissement : Profondeur CC nulle. Vérifiez les
453                     ↪ relations part_of.")
454
455         else:
456             print("  Fichier 'go-basic.obo' introuvable pour le test.")
457             print("  Veuillez placer le fichier dans le dossier courant ou
458                 ↪ modifier le chemin.")

```